Extended Computational Verification of the Ternary $(4k \pm 1)/3$ Collatz-Type Map up to 10^{11}: A 128-Bit Exhaustive Certification

Farhad Banazadeh
Independent Researcher
fn.bana@hotmail.com
ORCID: 0009-0004-7023-0298

10 September 2026

Abstract

This note extends the finite computational verification of a ternary Collatz-type map from 10^9 to 10^{11}. The map is

$$T(k) = \begin{cases} k/3, & k \equiv 0 \pmod 3, \\ (4k-1)/3, & k \equiv 1 \pmod 3, \\ (4k+1)/3, & k \equiv 2 \pmod 3. \end{cases}$$

A previous exhaustive computation verified that every starting value $1 \le n \le 10^9$ reaches the fixed point 1. The present computation examines every one of the $99,000,000,000$ additional starting values $10^9 < n \le 10^{11}$, using a certificate strategy that follows each orbit only until it reaches the previously verified base interval $[1, 10^9]$. Orbit states and peaks are stored in unsigned 128-bit arithmetic. Every tested start was certified, with zero 128-bit overflows. The maximum number of steps required to enter the base interval was 548, attained by $n = 58,528,894,046$. The largest observed value was $1,030,251,791,629,144,029,921$, reached from $n = 72,496,238,260$ at step 168. Combining the new certificate scan with the earlier base verification gives a finite exhaustive computational verification that every starting value $1 \le n \le 10^{11}$ reaches 1. This remains a finite computational result and is not a proof of global convergence.

1 Introduction

The ternary $(4k \pm 1)/3$ map was introduced and computationally studied in an earlier paper, where every starting value through 10^9 was reported to reach 1 [2]. The same paper recorded a longest stopping time of 540 steps, beginning at $972,526,420$, and a largest peak of $193,592,042,138,992,737$, beginning at $918,487,522$.

The purpose of the present note is deliberately narrower. No new global convergence theorem is claimed. Instead, the finite verification range is extended by two orders of magnitude, from 10^9 to 10^{11}, with a new C implementation that keeps the orbit state in unsigned 128-bit arithmetic. This change is important because trajectories in the enlarged interval can exceed the 64-bit unsigned range.

The computation is organized as a certificate extension. Since the interval $[1, 10^9]$ has already been exhaustively verified, a start $n > 10^9$ is certified as soon as its orbit reaches a value at most 10^9. Thus the new computation does not unnecessarily recompute the already verified tail of each trajectory.

2 The map and the certificate principle

For positive integers k, define

$$T(k) = \begin{cases} k/3, & k \equiv 0 \pmod 3, \\ (4k-1)/3, & k \equiv 1 \pmod 3, \\ (4k+1)/3, & k \equiv 2 \pmod 3. \end{cases} \tag{1}$$

Each branch is integral and positive. Also $T(1) = 1$.

Let

$$B = 10^9.$$

The earlier computation establishes the finite statement

$$\forall m \in [1, B], \qquad T^j(m) = 1 \quad \text{for some } j \geq 0. \tag{2}$$

For a new starting value $n > B$, define its entry time into the verified base interval by

$$\tau_B(n) = \min\{j \geq 0 : T^j(n) \leq B\},$$

when such a j exists. If the new program finds $\tau_B(n) < \infty$, then Eq. (2) immediately certifies that the full orbit of n eventually reaches 1. This is the only logical dependency of the extension on the previous computation.

3 128-bit implementation

The exhaustive program is written in C11 with POSIX threads. Starting values, interval endpoints, and aggregate counters fit in 64 bits, while the evolving orbit state and peak are represented by `unsigned __int128`. Four worker threads process fixed contiguous subintervals.

For residues 1 and 2, overflow is checked before evaluating $4k - 1$ or $4k + 1$. A trajectory is marked unresolved only if the next affine numerator cannot be represented in unsigned 128-bit arithmetic. The run also writes every such starting value to a separate overflow file. In the completed computation this count was zero and the overflow-start file was empty.

The exact command used for the final run was

```
./ternary_v3_u128 \
  --start 1000000001 \
  --end 100000000000 \
  --threads 4 \
  --mode certificate \
  --prefix scan_1e9_to_1e11_u128
```

The certificate threshold defaults to 10^9. In certificate mode, `reached_one=0` is expected: the program stops when the orbit first reaches the already verified base interval rather than continuing the tail to 1.

4 Exhaustive result

The final run reported

$$\boxed{99,000,000,000 \text{ tested}}, \qquad \boxed{99,000,000,000 \text{ certified}}, \qquad \boxed{0 \text{ unsigned-128 overflows}}.$$

The tested interval was exactly

$$1,000,000,001 \le n \le 100,000,000,000.$$

The wall-clock runtime recorded by the program was $45,956.016890$ seconds, approximately 12 hours 45 minutes 56 seconds, at an average rate of about $2,154,234$ starting values per second.

The aggregate branch counts before entry into the base interval were

Branch	Count
$k/3$	775,245,501,562
$(4k-1)/3$	775,235,824,695
$(4k+1)/3$	775,248,301,204
Total scanned steps	2,325,729,627,461

The three counts are each extremely close to one third of the scanned steps. This observation is descriptive only; it is not used as a proof of convergence.

5 New record trajectories

5.1 Longest descent to the verified base interval

The largest certificate entry time was

$$\boxed{\tau_{10^9}(58,528,894,046) = 548}.$$

The largest value encountered along this certificate segment was

$$30,589,364,851,501,665.$$

The quantity 548 is a steps-to-base record for the new interval, not a full stopping time to 1. Once the trajectory enters $[1, 10^9]$, convergence to 1 follows from the earlier exhaustive base computation.

5.2 Largest observed peak

The global peak in the extension scan was

$$\boxed{1,030,251,791,629,144,029,921},$$

obtained from

$$\boxed{n = 72,496,238,260}$$

at step 168. The certificate segment for this start contained 364 scanned steps before reaching the base interval.

This peak is about 5.32×10^3 times the largest peak reported in the earlier 10^9 computation. It also exceeds the maximum unsigned 64-bit integer $2^{64} - 1 \approx 1.8447 \times 10^{19}$ by a wide margin. This provides a concrete reason for using 128-bit orbit arithmetic in the enlarged scan.

6 Relation to the earlier 64-bit scan

An earlier 64-bit certificate implementation processed the same extension interval but reported 158 unresolved trajectories because their orbit values exceeded the representable unsigned 64-bit range. Those cases were computational overflows, not counterexamples. The 128-bit run reported zero overflows and certified all $99,000,000,000$ starts. Thus the previously unresolved cases disappear when the arithmetic range is enlarged.

The 64-bit and 128-bit implementations agree on the maximum steps-to-base record, 548 at $58,528,894,046$. The 128-bit run additionally reveals the much larger peak above, which cannot be represented by a 64-bit unsigned integer.

7 Combined finite verification through 10^{11}

The earlier result verifies every start in $[1, 10^9]$. The present result verifies that every start in $(10^9, 10^{11}]$ enters that base interval. Therefore the two computations combine to give the finite statement

$$\boxed{\forall n \in \mathbb{Z}_{>0},\ 1 \le n \le 10^{11} \implies T^j(n) = 1 \text{ for some } j \ge 0.} \tag{3}$$

This statement concerns exactly the finite interval shown. It does not establish that every positive integer reaches 1, does not rule out a nontrivial cycle entirely above 10^{11}, and does not rule out a divergent orbit beginning beyond the verified range.

8 Reproducibility

The computational package accompanying this note contains the final C source `ternary_v3_u128.c`, machine-readable and plain-text summaries of the 10^{11} extension scan, and the empty file recording genuine 128-bit overflow starts. The summaries record the interval, threshold, thread count, tested and certified counts, branch totals, runtime, maximum steps-to-base record, and maximum peak record.

The program can be compiled on a compatible C11 system with Clang using, for example,

```
clang -O3 -march=native -std=c11 -pthread \
  ternary_v3_u128.c -o ternary_v3_u128
```

The reported exhaustive run was performed on a MacBook Pro using four worker threads. Because `unsigned __int128` is a compiler extension rather than an ISO C fixed-width type, reproduction requires a compiler that supports it, such as the Clang/GCC toolchains used for this computation.

9 Conclusion

The finite computational verification of the ternary $(4k \pm 1)/3$ map has been extended from 10^9 to 10^{11}. Every one of the 99 billion additional starting values was certified to enter the previously verified base interval, with no unsigned-128 overflow. The largest steps-to-base value was 548, starting at $58,528,894,046$, and the largest observed peak was $1,030,251,791,629,144,029,921$, starting at $72,496,238,260$.

Together with the earlier exhaustive base scan, these results computationally verify that every starting value through 10^{11} reaches 1. The result is substantial finite evidence for the ternary stop-one conjecture, but the global conjecture remains open.

Data and code availability

The source code and result summaries used for the extension are supplied with the computational archive associated with this paper. The author's research repository is https://github.com/FarhadBanazadeh/sequence-computations.

References

[1] J. C. Lagarias, "The $3x + 1$ problem and its generalizations," *The American Mathematical Monthly*, 92(1), 3–23, 1985. DOI: 10.2307/2322189.

[2] F. Banazadeh, "A Ternary $(4k \pm 1)/3$ Collatz-Type Map: Computational Verification up to 10^9," 2026. Associated Zenodo records: 10.5281/zenodo.22195651 and 10.5281/zenodo.22195652.